

Deep Learning

Unit-5

Deep reinforcement learning

Deep reinforcement learning combines artificial neural networks with a framework of reinforcement learning that helps software agents learn how to reach their goals. That is, it unites function approximation and target optimization, mapping states and actions to the rewards they lead to.

Reinforcement learning refers to goal-oriented algorithms, which learn how to achieve a complex objective (goal) or how to maximize along a particular dimension over many steps; for example, they can maximize the points won in a game over many moves. Reinforcement learning algorithms can start from a blank slate, and under the right conditions, achieve superhuman performance.

Reinforcement algorithms that incorporate deep neural networks can beat human experts playing numerous Atari video games, Starcraft II and Dota-2.

While that may sound trivial to non-gamers, it's a vast improvement over reinforcement learning's previous accomplishments, and the state of the art is progressing rapidly.

Reinforcement learning solves the hard problem of correlating immediate actions with the delayed outcomes they produce. Like humans, reinforcement learning algorithms sometimes have to wait to see the fruit of their decisions. They operate in a delayed-return environment, where it can be difficult to understand which action leads to which outcome over many time steps.

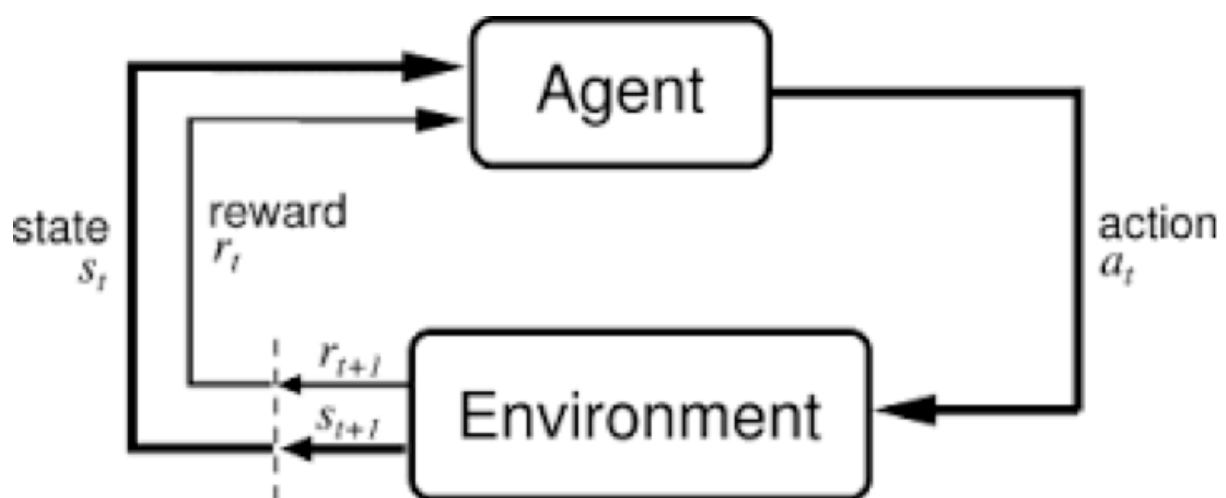
Reinforcement learning can be understood through the concepts of agents, environments, states, actions and rewards, all of which we'll explain below. Capital letters tend to denote sets of things, and lower-case letters denote a specific instance of that thing; e.g. A is all possible actions, while a is a specific action contained in the set.

- Agent: An **agent** takes actions; for example, a drone making a delivery, or Super Mario navigating a video game. The algorithm is the agent. It may be helpful to consider that in life, the agent is you.¹
- Action (A): A is the set of all possible moves the agent can make. An **action** is almost self-explanatory, but it should be noted that agents usually choose from a list of discrete, possible actions. In video games, the list might include running right or left, jumping high or low,

crouching or standing still. In the stock markets, the list might include buying, selling or holding any one of an array of securities and their derivatives. When handling aerial drones, alternatives would include many different velocities and accelerations in 3D space.

- Discount factor: The **discount factor** is multiplied by future rewards as discovered by the agent in order to dampen these rewards' effect on the agent's choice of action. Why? It is designed to make future rewards worth less than immediate rewards; i.e. it enforces a kind of short-term hedonism in the agent. Often expressed with the lower-case Greek letter gamma: γ . If γ is .8, and there's a reward of 10 points after 3 time steps, the present value of that reward is $0.8^3 \times 10$. A discount factor of 1 would make future rewards worth just as much as immediate rewards. We're fighting against delayed gratification here.
- Environment: The world through which the agent moves, and which responds to the agent. The environment takes the agent's current state and action as input, and returns as output the agent's reward and its next state. If you are the agent, the environment could be the laws of physics and the rules of society that process your actions and determine the consequences of them.
- State (S): A **state** is a concrete and immediate situation in which the agent finds itself; i.e. a specific place and moment, an instantaneous configuration that puts the agent in relation to other significant things such as tools, obstacles, enemies or prizes. It can be the current situation returned by the environment, or any future situation. Were you ever in the wrong place at the wrong time? That's a state.
- Reward (R): A **reward** is the feedback by which we measure the success or failure of an agent's actions in a given state. For example, in a video game, when Mario touches a coin, he wins points. From any given state, an agent sends output in the form of actions to the environment, and the environment returns the agent's new state (which resulted from acting on the previous state) as well as rewards, if there are any. Rewards can be immediate or delayed. They effectively evaluate the agent's action.
- Policy (π): The **policy** is the strategy that the agent employs to determine the next action based on the current state. It maps states to actions, the actions that promise the highest reward.

- Value (V): The expected long-term return with discount, as opposed to the short-term reward R . $V\pi(s)$ is defined as the expected long-term return of the current state under policy π . We discount rewards, or lower their estimated value, the further into the future they occur. See *discount factor*. And remember Keynes: “In the long run, we are all dead.” That’s why you discount future rewards. It is useful to distinguish
- Q-value or action-value (Q): **Q-value** is similar to Value, except that it takes an extra parameter, the current action a . $Q\pi(s, a)$ refers to the long-term return of an action taking action a under policy π from the current state s . Q maps state-action pairs to rewards. Note the difference between Q and policy.
- Trajectory: A sequence of states and actions that influence those states. From the Latin “to throw across.” The life of an agent is but a ball tossed high and arching through space-time unmoored, much like humans in the modern world.



Reinforcement learning judges actions by the results they produce. It is goal oriented, and its aim is to learn sequences of actions that will lead an agent to achieve its goal, or maximize its objective function.

Temporal Difference Learning:

Temporal Difference Learning, also known as TD-Learning, is a method for computing the long term utility of a pattern of behavior from a series of intermediate rewards (Sutton, [1984](#), [1988](#), [1998](#)). It uses differences between successive utility estimates as a feedback signal for learning. The Temporal Differencing approach to model-free [reinforcement learning](#) was introduced by, and is often associated with, R.S. Sutton. It has ties to both the artificial

intelligence and psychological theories of reinforcement learning as well as [dynamic programming](#)

Policy gradient methods

Policy gradient methods are a type of [reinforcement learning](#) techniques that rely upon optimizing parametrized policies with respect to the expected return (long-term cumulative [reward](#)) by gradient descent. They do not suffer from many of the problems that have been marring traditional [reinforcement learning](#) approaches such as the lack of guarantees of a value function, the intractability problem resulting from uncertain state information and the [complexity](#) arising from continuous states & actions.

[Reinforcement learning](#) is probably the most general framework in which reward-related learning problems of animals, humans or machine can be phrased. However, most of the methods proposed in the reinforcement learning community are not yet applicable to many problems such as robotics, motor control, etc. This inapplicability may result from problems with uncertain state information. Most traditional [reinforcement learning](#) methods have no convergence guarantees and there exist even divergence examples. Continuous states and actions in high dimensional spaces cannot be treated by most off-the-shelf [reinforcement learning](#) approaches.

Policy gradient methods differ significantly as they do not suffer from these problems in the same way. For example, uncertainty in the state might degrade the performance of the policy (if no additional state estimator is being used) but the [optimization](#) techniques for the policy do not need to be changed.

Continuous states and actions can be dealt with in exactly the same way as discrete ones while, in addition, the learning performance is often increased.

Convergence at least to a local optimum is guaranteed.

The advantages of policy gradient methods for real world applications are numerous. Among the most important ones are that the policy representations can be chosen so that it is meaningful for the task and can incorporate domain knowledge, that often fewer parameters are needed in the learning process than in value-function based approaches and that there is a variety of different [algorithms](#) for policy gradient estimation in the literature which have a rather strong theoretical underpinning. Additionally, policy gradient methods can be used either model-free or model-based as they are a generic formulation.

Of course, policy gradients are not the salvation to all problems but also have significant problems. They are by definition on-policy (note that tricks like importance sampling can slightly alleviate this problem) and need to forget data very fast in order to avoid the introduction of a bias to the gradient estimator.

Hence, the use of sampled data is not very efficient. In tabular representations, value function methods are guaranteed to converge to a global maximum while policy gradients only converge to a local maximum and there may be many maxima in discrete problems. Policy gradient methods are often quite demanding to apply, mainly because one has to have considerable knowledge about the system one wants to control to make reasonable policy definitions. Finally, policy gradient methods always have an open parameter, the learning rate, which may decide over the order of magnitude of the speed of convergence, these have led to new approaches inspired by expectation-maximization

Approaches to Policy Gradient Estimation

Finite-difference Methods

Finite-difference methods are among the oldest policy gradient approaches; The policy parameterization is varied I times by small increments $\Delta\theta_i, i=1 \dots I$ and for each policy parameter variation $\theta_h + \Delta\theta_i$ roll-outs (or trajectories) are performed which generate estimates $\Delta J^i \approx J(\theta_h + \Delta\theta_i) - J_{ref}$ of the expected return. There are different ways of choosing the reference value J_{ref} , e.g. forward-difference estimators with $J_{ref} = J(\theta_h)$ and central-difference estimators with $J_{ref} = J(\theta_h - \Delta\theta_i)$. The policy gradient estimate $g_{FD} \approx \nabla_{\theta} J|_{\theta=\theta_h}$ can be estimated by regression yielding

$$g_{FD} = (\Delta \Theta^T \Delta \Theta)^{-1} \Delta \Theta^T \Delta J^{\wedge},$$

Advantages of this approach: Finite-difference methods require very little skill and can usually be implemented out of the box. They work both with stochastic and deterministic policies without any change. It is highly efficient in simulation with a set of common histories of random numbers and on totally deterministic systems.

Disadvantages of this approach: The perturbation of the parameters is a very difficult problem often with disastrous impact on the learning process when the system goes [unstable](#). In the presence of noise on a real system, the gradient estimate error decreases much slower than for the following methods. Performance depends highly on the chosen policy parametrization.

Natural Policy Gradients

One of the main reasons for using policy gradient methods is that we intend to do just a small change $\Delta\theta$ to the policy π_{θ} while improving the policy. However, the meaning of small is ambiguous. When using the Euclidian metric of $\Delta\theta^T \Delta\theta$, then the gradient is different for every

parameterization θ of the policy π_θ even if these parameterizations are related to each other by a linear transformation (Kakade, 2002). This problem poses the question of how we can measure the closeness between the current policy and the updated policy based upon the distribution of the paths generated by each of these. In statistics, a variety of distance measures for the closeness of two distributions (e.g., $p_\theta(\tau)$ and $p_{\theta+\Delta\theta}(\tau)$) have been suggested, e.g., the Kullback-Leibler divergence $d_{KL}(p_\theta, p_{\theta+\Delta\theta})$, the Hellinger distance d_{HD} and others (Su & Gibbs, 2002). Many of these distances (e.g., the previously mentioned ones) can be approximated by its second order Taylor expansion, i.e., by

$$d_{KL}(p_\theta, p_{\theta+\Delta\theta}) \approx \Delta\theta^T \mathbf{F}_\theta \Delta\theta,$$

Advantages of this approach: Natural policy gradients differ in a deciding aspect from both finite difference gradients and regular likelihood ratio gradients, i.e., they are independent from the choice of policy parametrization if the choices have the same representational power. As a result, they can be an order of magnitude faster than the regular gradient. They also profit from most other advantages of the regular policy gradients.

Disadvantages of this approach: In comparison to the regular policy gradient, there are three disadvantages: first, the matrix inversion in the gradient estimators may be numerically brittle and may scale worse (note that there are tricks to alleviate this problem). Second, if we can find a special policy parametrization that trivializes a problem, the natural policy gradient may not make use of it. Third, the natural policy gradient estimators are often much harder to implement.

Q-learning

What is q-learning?

Q-learning is an off policy reinforcement learning algorithm that seeks to find the best action to take given the current state. It's considered off-policy because the q-learning function learns from actions that are outside the current policy, like taking random actions, and therefore a policy isn't needed. More specifically, q-learning seeks to learn a policy that maximizes the total reward.

Create a q-table

When q-learning is performed we create what's called a *q-table* or matrix that follows the shape of [state, action] and we initialize our values to zero. We then update and store our *q-values* after an episode. This q-table becomes a reference table for our agent to select the best action based on the q-value.

```
import numpy as np
# Initialize q-table values to 0
Q = np.zeros((state_size, action_size))
```

Q-learning and making updates

The next step is simply for the agent to interact with the environment and make updates to the state action pairs in our q-table `Q[state, action]`.

Taking Action: Explore or Exploit

An agent interacts with the environment in 1 of 2 ways. The first is to use the q-table as a reference and view all possible actions for a given state. The agent then selects the action based on the max value of those actions. This is known as ***exploiting*** since we use the information we have available to us to make a decision.

The second way to take action is to act randomly. This is called ***exploring***. Instead of selecting actions based on the max future reward we select an action at random. Acting randomly is important because it allows the agent to explore and discover new states that otherwise may not be selected during the exploitation process. You can balance exploration/exploitation using epsilon (ϵ) and setting the value of how often you want to explore vs exploit. Here's some rough code that will depend on how the state and action space are setup.

Updating the q-table

The updates occur after each step or action and ends when an episode is done. Done in this case means reaching some terminal point by the agent. A terminal state for example can be anything like landing on a checkout page, reaching the end of some game, completing some desired objective, etc. The agent will not learn much after a single episode, but eventually with enough exploring (steps and episodes) it will converge and learn the optimal q-values or q-star (Q^*).

Here are the 3 basic steps:

1. Agent starts in a state (s_1) takes an action (a_1) and receives a reward (r_1)
2. Agent selects action by referencing Q-table with highest value (max) **OR** by random (epsilon, ϵ)
3. Update q-values

Learning Rate: lr or learning rate, often referred to as *alpha* or α , can simply be defined as how much you accept the new value vs the old value. Above we are taking the difference between new and old and then multiplying that value by the learning rate. This value then gets added to our previous q-value which essentially moves it in the direction of our latest update.

Gamma: γ or γ is a discount factor. It's used to balance immediate and future reward. From our update rule above you can see that we apply the

discount to the future reward. Typically this value can range anywhere from 0.8 to 0.99.

Reward: reward is the value received after completing a certain action at a given state. A reward can happen at any given time step or only at the terminal time step.

Max: `np.max()` uses the numpy library and is taking the maximum of the future reward and applying it to the reward for the current state. What this does is impact the current action by the possible future reward. This is the beauty of q-learning. We're allocating future reward to current actions to help the agent select the highest return action at any given state.

Deep Q-Learning

The process of Q-Learning creates an exact matrix for the working agent which it can "refer to" to maximize its reward in the long run. Although this approach is not wrong in itself, this is only practical for very small environments and quickly loses its feasibility when the number of states and actions in the environment increases.

The solution for the above problem comes from the realization that the values in the matrix only have relative importance ie the values only have importance with respect to the other values. Thus, this thinking leads us to **Deep Q-Learning** which uses a deep neural network to approximate the values. This approximation of values does not hurt as long as the relative importance is preserved.

The basic working step for Deep Q-Learning is that the initial state is fed into the neural network and it returns the Q-value of all possible actions as an output.

Deep Q-Learning: A Neural Network maps input states to (action, Q-value) pairs

The Deep Q-Network Algorithm

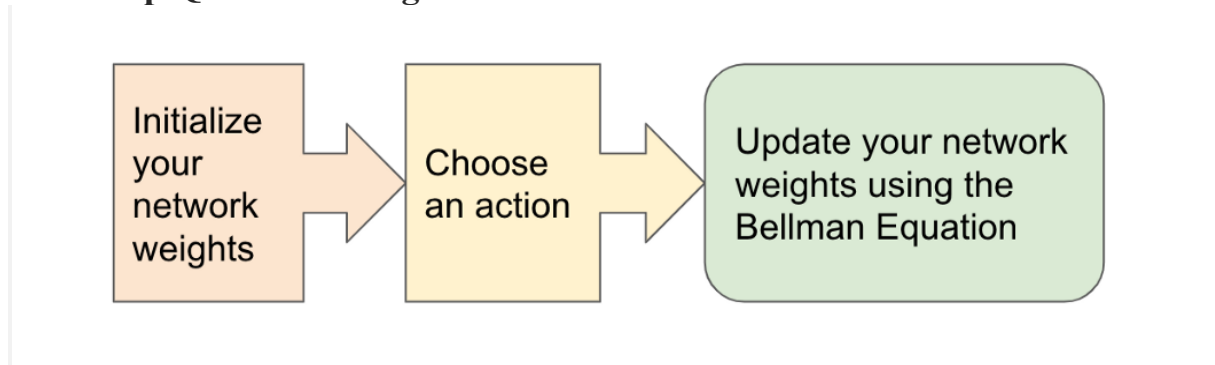


Figure 5: The Deep Q-Network Algorithm (Image by Author)

1. Initialize your Main and Target neural networks
2. Choose an action using the Epsilon-Greedy Exploration Strategy
3. Update your network weights using the Bellman Equation

4a. Initialize your Target and Main neural networks

A core difference between Deep Q-Learning and Vanilla Q-Learning is the implementation of the Q-table. Critically, Deep Q-Learning replaces the regular Q-table with a neural network. Rather than mapping a state-action pair to a q-value, a neural network maps input states to (action, Q-value) pairs.

One of the interesting things about Deep Q-Learning is that the learning process uses 2 neural networks. These networks have the same architecture but different weights. Every N steps, the weights from the **main network** are copied to the **target network**. Using both of these networks leads to more stability in the learning process and helps the algorithm to learn more effectively. In our

implementation, the main network weights replace the target network weights every 100 steps.

How to map States to (Action, Q-value) pairs

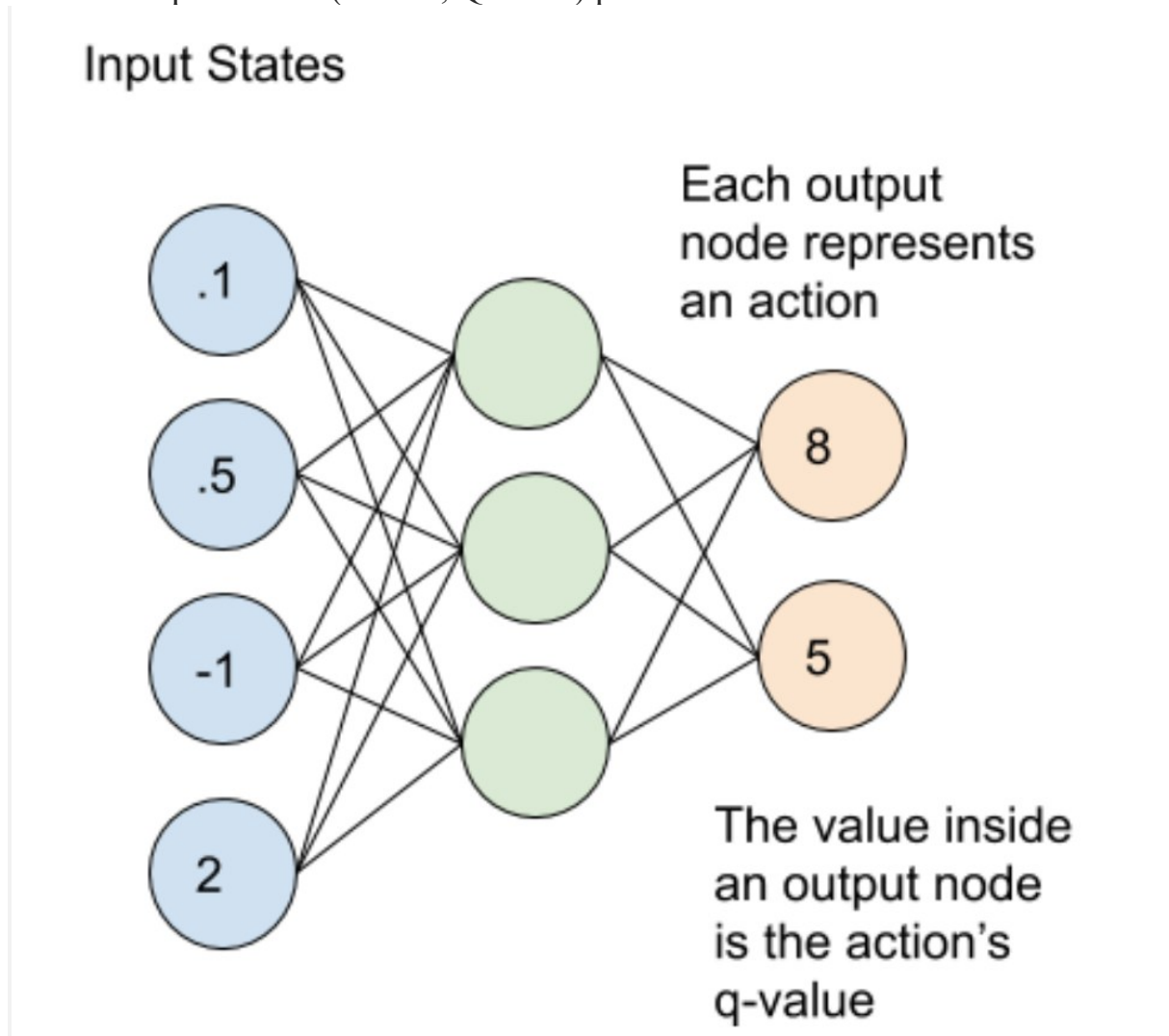


Figure 6: A neural network mapping an input state to its corresponding (action, q-value) pair (Image by Author)

The main and target neural networks map input states to an (action, q-value) pair. In this case, each output node (representing an action) contains the action's q-value as a floating point number. Note that the output nodes do not represent a probability distribution so they will not add up to 1. For the example above, one action has a Q-value of 8 and the other action has a Q-value of 5.

4b. Choose an action using the Epsilon-Greedy Exploration Strategy

In the Epsilon-Greedy Exploration strategy, the agent chooses a random action with probability **epsilon** and exploits the best known action with probability **1 — epsilon**.

4c. Update your network weights using the Bellman Equation

After choosing an action, it's time for the agent to perform the action and update the Main and Target networks according to the Bellman equation. Deep Q-Learning agents use Experience Replay to learn about their environment and update the Main and Target networks.

To summarize, the **main network** samples and trains on a batch of past experiences every 4 steps. The main network weights are then copied to the **target network** weights every 100 steps.